

Week 3 Pre-Class Study Guide: Decision Trees & Ensembles

Time Required: 30 minutes **Complete By:** Before Week 3 live session

Learning Objectives

By completing this pre-class work, you will:

- ☐ Understand how decision trees split data using binary questions
 - ☐ Grasp the intuition behind ensemble methods (Random Forests, XGBoost)
 - ☐ Recognize when to use tree-based algorithms vs linear models
 - ☐ Be familiar with key terminology (bagging, boosting, feature importance)
 - ☐ Come to class ready to compare algorithms systematically
-

Required Videos (28 minutes total)

Video 1: StatQuest - Decision Trees (18 minutes)

Link: <https://www.youtube.com/watch?v=7VeUPuFGJHk>

What to watch for:

- 0:00-3:00: How trees ask yes/no questions to classify data
- 3:00-8:00: Information gain and how trees decide which question to ask
- 8:00-13:00: Gini impurity (how “mixed” is a group?)
- 13:00-18:00: Overfitting risk and pruning

Key Questions to Answer While Watching:

1. What is a decision tree doing at each node?
2. How does a tree decide which feature to split on first?
3. Why can decision trees overfit if we let them grow too deep?

Your Notes:

[Space for your notes]

Video 2: StatQuest - Random Forests Part 1 (10 minutes)

Link: https://www.youtube.com/watch?v=J4Wdy0Wc_xQ

What to watch for:

- 0:00-3:00: Ensemble methods - combining multiple models
- 3:00-6:00: Bootstrap sampling (how each tree sees different data)
- 6:00-10:00: Why Random Forests are more robust than single trees

Key Questions to Answer While Watching:

1. What is “bagging” (bootstrap aggregating)?
2. How does Random Forest reduce overfitting compared to a single tree?
3. Why use 100 trees instead of just one really good tree?

Your Notes:

[Space for your notes]

Optional Reading (10 minutes)

Optional Visual Tutorial: R2D3 Visual Intro to ML

Link: <http://www.r2d3.us/visual-intro-to-machine-learning-part-1/>

This is beautiful interactive visualization of decision trees and Random Forests. Highly recommended if you're a visual learner!

What you'll see:

- Visual representation of how decision trees partition data
- How Random Forests combine multiple trees
- Interactive examples with real data

Why it's optional: The videos above cover the concepts; this provides visual reinforcement.

Pre-Class Self-Check Quiz

Test your understanding before class. **Don't worry if you can't answer everything perfectly** - we'll cover it all in detail during the live session!

Question 1

What do decision trees do at each node?

- ☐ a) Calculate mean values
- ☐ b) Ask a yes/no question to split data into two groups
- ☐ c) Apply linear regression
- ☐ d) Randomly assign classes

Click to reveal answer

Answer: b) Decision trees ask binary (yes/no) questions at each node to split data into purer groups. Each split partitions the data based on a feature threshold (e.g., "Is age > 30?").

Question 2

How does a Random Forest make predictions?

- ☐ a) Uses the single best tree
- ☐ b) Takes the average/vote of many trees
- ☐ c) Picks a random tree each time
- ☐ d) Combines all trees into one big tree

Click to reveal answer

Answer: b) Random Forest trains many decision trees (typically 100+) on different bootstrap samples of the data, then takes the majority vote (classification) or average (regression) of all trees' predictions. This is called "ensemble learning."

Question 3

What is the main risk of deep decision trees?

- ☐ a) Too slow to train
- ☐ b) Can't handle categorical features
- ☐ c) Overfitting - memorizing training data instead of learning patterns
- ☐ d) Requires too much memory

Click to reveal answer

Answer: c) Deep trees with many splits can create very specific rules that memorize the training data (achieving near-perfect training accuracy) but fail to generalize to new data. This is overfitting, and we prevent it by limiting tree depth with the `max_depth` parameter.

Question 4

What is “bagging” in Random Forests?

- ☐ a) Removing outliers from the dataset
- ☐ b) Bootstrap Aggregating - training trees on random samples with replacement
- ☐ c) Putting features into bags
- ☐ d) A data cleaning technique

Click to reveal answer

Answer: b) Bagging = Bootstrap Aggregating. Each tree in a Random Forest is trained on a random sample (with replacement) of the training data. This creates diversity among trees, which makes the ensemble more robust than any single tree.

Question 5

Which algorithm is best when you need to EXPLAIN predictions to regulators?

- ☐ a) XGBoost (highest accuracy)
- ☐ b) Random Forest (100 trees)
- ☐ c) Single Decision Tree (most interpretable)
- ☐ d) Neural Network

Click to reveal answer

Answer: c) Single decision trees are the most interpretable tree-based model - you can visualize the entire tree and explain exactly why a prediction was made (“The model predicted high risk because income < \$30K AND debt > \$50K”). Random Forests and XGBoost are more accurate but harder to explain to non-technical audiences.

Score Your Pre-Class Quiz

How many did you get right?

- **5/5:** Excellent! You're well-prepared for Week 3.
- **3-4/5:** Good foundation. Pay extra attention to concepts you missed during class.
- **1-2/5:** That's okay! The videos are dense. The live session will clarify everything.

- **0/5:** No problem! Come to class ready to learn. The instructor will explain from scratch.
-

What to Expect in the Live Session

Segment 1 (0:00-0:15): Week 2 Recap

- Quick review of classification metrics
- Questions from logistic regression homework

Segment 2 (0:15-0:45): Decision Trees Deep Dive

- You'll SEE a decision tree working before learning theory
- “20 Questions Game” analogy for how trees split data
- Information gain intuition (no heavy math!)
- Overfitting warning and `max_depth` parameter

Segment 3 (0:45-1:10): Random Forests

- Ensemble methods: “wisdom of crowds”
- Why 100 mediocre trees beat one perfect tree
- Feature importance extraction
- When to use Random Forest (robust default algorithm)

Segment 4 (1:10-1:25): BREAK

Segment 5 (1:25-1:55): XGBoost

- Boosting vs bagging (sequential vs parallel)
- Why XGBoost wins Kaggle competitions
- Key hyperparameters: `learning_rate`, `n_estimators`, `max_depth`
- Installation and usage (new library!)

Segment 6 (1:55-2:30): Live Coding - Systematic Algorithm Comparison

- **You'll code along!** Adult Income dataset
- Implement all three algorithms (Decision Tree, Random Forest, XGBoost)
- Calculate metrics for each
- Compare feature importance across models
- Create comparison table
- Make informed algorithm recommendation

Segment 7 (2:30-2:50): Pair Programming - “Algorithm Showdown”

- Your turn to practice with a partner
- Explain tree prediction paths
- Compare feature importance interpretations
- Choose best algorithm for business scenarios

Segment 8 (2:50-3:00): Wrap-up

- When to use each algorithm
 - How to explain tree models to stakeholders
 - Preview of Week 4 (cross-validation and hyperparameter tuning)
-

What to Bring to Class

Required: - [] Laptop with Python and Jupyter installed (or Google Colab access) - [] XGBoost library installed (see installation instructions below) - [] This study guide (your notes) - [] Curiosity about how

trees work!

Optional but helpful: - [] Notebook/paper for additional notes - [] Second screen (if virtual) to have docs open alongside code

IMPORTANT: XGBoost Installation

Week 3 introduces XGBoost, a powerful library that's NOT included in standard sklearn. You need to install it separately.

Installation Instructions

Option 1: Using pip (most common)

```
pip install xgboost
```

Option 2: Using conda (if you use Anaconda)

```
conda install -c conda-forge xgboost
```

Test Your Installation

Run this in Python or Jupyter to verify XGBoost is installed:

```
import xgboost as xgb
print(f"XGBoost version: {xgb.__version__}")
```

Expected output: XGBoost version: 2.x.x (any 2.x version is fine)

Troubleshooting

If installation fails: 1. Try updating pip: `pip install --upgrade pip` 2. Then retry: `pip install xgboost` 3. If still failing, install from conda-forge (see Option 2) 4. **Last resort:** Tell instructor at start of class - we have a backup plan

Common error: "Microsoft Visual C++ required" (Windows) - **Solution:** Install Visual C++ Build Tools or use conda instead

Don't stress if XGBoost won't install! We can work around it during class, but try to install it beforehand.

Tips for Success

Before the Live Session

1. Watch videos in order (Decision Trees, then Random Forests)
2. Take notes while watching - pause and rewind as needed
3. Complete the self-check quiz
4. Install XGBoost and verify it works
5. Review Week 2 classification metrics (accuracy, precision, recall, F1)

During the Live Session

1. Watch the decision tree demo carefully - it's the foundation for everything else
2. Ask questions when the "20 Questions" analogy is explained
3. Take notes on algorithm tradeoffs (interpretability vs accuracy)
4. Code along during Segment 6 - don't just watch!

5. Engage actively during pair programming

After the Live Session

1. Complete the post-class homework (Titanic dataset with all three algorithms)
 2. Review the comparison methodology - you'll use it in future projects
 3. Compare your solution to the provided solution notebook
-

Additional Resources (Optional)

If you want to dive deeper BEFORE class (completely optional):

Interactive Tutorials:

- XGBoost Documentation: <https://xgboost.readthedocs.io/en/latest/>
- sklearn Decision Trees: <https://scikit-learn.org/stable/modules/tree.html>
- sklearn Random Forests: <https://scikit-learn.org/stable/modules/ensemble.html#forests-of-randomized-trees>

Visual Explanations:

- Decision Tree Visualization Tool: <https://explained.ai/decision-tree-viz/>
- Random Forest Explained Visually: <https://mlu-explain.github.io/random-forest/>

Reading:

- Appendix B: Algorithm Comparison Template - Preview the systematic comparison workflow
-

Week 3 Key Terminology Preview

Get familiar with these terms - you'll use them all session!

Decision Trees:

- **Node:** Point where tree asks a yes/no question
- **Leaf:** Terminal node with final prediction
- **Information Gain:** How much a split reduces uncertainty
- **Gini Impurity:** Measure of how "mixed" a group is (0 = pure, 0.5 = perfectly mixed)
- **max_depth:** Hyperparameter limiting how many questions tree can ask

Random Forests:

- **Ensemble:** Combining multiple models for better predictions
- **Bagging:** Bootstrap Aggregating - training trees on random samples
- **n_estimators:** Number of trees in the forest (typically 100)
- **Feature Importance:** Which features contribute most to predictions
- **Out-of-Bag (OOB) Score:** Validation using samples not used in training

XGBoost:

- **Boosting:** Sequential ensemble - each tree learns from previous trees' mistakes
- **Gradient Boosting:** Specific boosting algorithm XGBoost uses
- **learning_rate:** How much each tree contributes (lower = more conservative)
- **Early Stopping:** Stop training when performance stops improving

Comparison Concepts:

- **Interpretability:** How easily you can explain predictions to non-experts

- **Overfitting:** Memorizing training data instead of learning patterns
 - **Feature Engineering:** Creating new features that trees can split on
 - **Systematic Comparison:** Comparing algorithms on same dataset with same metrics
-

Pre-Class Completion Checklist

Before the live session, make sure you:

- ☐ Watched StatQuest Decision Trees (18 min)
- ☐ Watched StatQuest Random Forests Part 1 (10 min)
- ☐ (Optional) Explored R2D3 visual intro
- ☐ Completed the 5-question self-check quiz
- ☐ Installed XGBoost and verified it works
- ☐ Reviewed Week 2 classification metrics
- ☐ Tested your Python/Jupyter environment
- ☐ Reviewed learning objectives

Estimated total time: 30 minutes (40 if including optional visual tutorial)

Questions Before Class?

If anything is unclear, don't stress! The live session will bring it all together.

Week 3 Big Picture

What makes Week 3 different from Weeks 1-2:

- **Week 1 (Linear Regression):** ONE algorithm, focus on ML pipeline
- **Week 2 (Logistic Regression):** ONE algorithm, focus on classification metrics
- **Week 3 (Trees & Ensembles):** THREE algorithms, focus on SYSTEMATIC COMPARISON

Why this matters:

In real ML projects, you don't know which algorithm will work best beforehand. Week 3 teaches you to:

1. Train multiple algorithms on the same problem
2. Compare them systematically using consistent metrics
3. Choose the best one based on business constraints
4. Explain your choice to stakeholders

This is the workflow data scientists use in practice!

See you in class!

Last Updated: 2026-01-24